

NHN NAN 2026 · 게임 × AI 해커톤 사전과제

# AI 활용 기술 문서

「최후의 벽, 최후의 사람」 — 도구 · 프롬프트 · 검증 기록

개인 참가 · ry-eon

플레이 <https://ry-eon.github.io/NAN2026-NHN-Game-X-AI-Hackathon/>

소스 · 커밋 기록 <https://github.com/ry-eon/NAN2026-NHN-Game-X-AI-Hackathon>

2026-08-10

## 0. 한 줄 요지

생성이 아니라 보증.

AI가 게임을 만들었다는 사실은 이 대회에서 흔한 주장이 될 것이다. 우리가 제출하는 것은 그 주장이 아니라, **만든 것이 게임으로 성립함을 시스템이 증명한 기록**이다. 결정론 시뮬레이션 위에서 봇 4등급이 판을 직접 플레이해 판정하고, 통과한 것만 출고한다. 반려 사유와 실패 데이터를 그대로 공개한다 — 그것이 검증기가 진짜라는 유일한 증거다.

## 1. 사용한 AI 도구

| 도구                                   | 어디에 썼나                                                | 비고                |
|--------------------------------------|-------------------------------------------------------|-------------------|
| Claude (Anthropic) — Claude Code CLI | 설계 논의 · 코드 작성 · 봇 정책 · 밸런스 스왑 스크립트 · 검증 결과 해석 · 문서 작성 | 이 프로젝트의 코드·문서 대부분 |
| Claude — 브라우저 자동화                    | 실기기 크롬을 몰아 스크린샷·콘솔·런타임 측정 (§8)                        | 자기 결과물 확인용        |

**쓰지 않은 것도 기록해 둔다.** 이미지 생성·3D 모델 생성·음성/음악 생성 AI를 일절 쓰지 않았다. 캐릭터·괴수·보스·성체는 전부 코드로 만든 **절차 지오메트리**이고, 효과음 18종은 **WebAudio 합성**이다 (오디오 파일 0개). 외부 에셋은 CC0 텍스처·HDRi뿐이다 (§9). 런타임에 AI API를 호출하는 곳은 **없다** — 심사 요건(유료 API·라이선스 금지)에 맞춰 AI의 산출물은 전부 **개발 시점에 코드로 굳혔다**.

## 2. AI를 어디에 배치했는가

이 프로젝트에서 AI(Claude)는 "코드를 대신 쳐주는 손"이 아니라 **세 자리에 나뉘어 배치**됐다. 각 자리는 책임이 다르고, 서로를 검사한다.

| 자리    | 책임                               | 사람(디렉터)이 하는 일                     |
|-------|----------------------------------|-----------------------------------|
| 설계·구현 | sim 규칙, 렌더, 봇 정책을 코드로 옮긴다        | 무엇을 만들지 결정, 프레이밍 반려, 우선순위         |
| 검증    | 봇이 판을 플레이하고 판정 기준으로 반려/통과를 낸다    | <b>판정 기준 자체를 정한다</b> (= 게임 설계 주장) |
| 자체 점검 | 실기기 브라우저를 몰아 스크린샷·콘솔로 자기 결과물을 확인 | 체감(조작감·리듬)은 사람만 판정 가능             |

### AI를 런타임이 아니라 파이프라인에 둔 이유

이 프로젝트에서 **AI를 어디에 둘지**가 가장 큰 아키텍처 결정이었다. 플레이 중 LLM이 대화하거나 콘텐츠를 즉석 생성하는 구조를 택하지 않았고, 근거는 셋이다.

|         | 근거                                                                                                                                                       |
|---------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| ①<br>보증 | 실시간 생성은 <b>매 판이 다르다</b> . 재현이 안 되면 검증도 못 하고, 그러면 \$0의 문장("만든 것이 게임으로 성립함을 시스템이 증명한 다")이 성립하지 않는다. <b>결정론을 포기하는 순간 보증도 함께 사라진다</b> — 이 프로젝트에서는 이것이 결정적이었다 |
| ②<br>운영 | 런타임 호출은 <b>플레이 1회마다 과금</b> 이다. 많이 즐길수록 적자가 되는 구조라, 충분한 인프라와 단가가 전제된 조직에서만 서비스로 성립한다. 콘텐츠 생성만이 아니라 <b>운영까지 놓고</b> 고른 자리다                                  |
| ③<br>보안 | 상호작용은 <b>컨텍스트 유지</b> 가 전제다. 그 방향으로 갔다면 유저 입력과 플레이 상태를 외부 상용 API로 계속 보내야 하고, 유저 정보 취급·보관·위탁 문제가 따라온다. 지금 이 게임은 <b>외부로 나가는 데이터가 0</b> 이다                   |

그래서 AI는 게임 안이 아니라 **게임을 만드는 파이프라인**에 있다: 생성(AI) → 검증(봇 판정) → 통과분만 정적 빌드로 출고 .

이 구조라서 **빠르게 만들고 빠르게 버릴 수 있었다**. 밸런스 후보 13종·편성 4종·스킬 사거리 조합 18종을 봇으로 돌려 숫자로 골랐다(\$5). 사람이 손으로 플레이해 고르는 방식으로는 불가능한 양이고, 반려된 안들이 그대로 이 문서의 증거가 됐다.

핵심은 **검증이 구현과 분리돼 있고, 검증이 구현을 반려할 수 있다**는 점이다. 아래 \$5에서 실제로 검증이 설계를 뒤집은 기록을 제시한다.

### 3. 시스템 구조

|                    |                                                                                                     |
|--------------------|-----------------------------------------------------------------------------------------------------|
| siege/sim/world.ts | 순수 TypeScript 결정론 시뮬레이션. three.js·DOM을 모른다.<br>고정 30Hz 틱, 시드 RNG(mulberry32)만 사용. Math.random() 금지. |
| siege/bots/        | 봇 정책(policy) · 러너(runner) · 판정(verify)                                                              |
| client/src3d/      | Three.js 렌더러. sim을 구독만 한다. 게임 로직 없음.                                                                |

#### 3-1. 이 분리가 논지의 전제다

봇과 플레이어가 **같은 sim을 같은 입력 타입으로** 돌린다. 봇 전용 뒷문이 없다.

```
// 봇 정책의 시그니처 — 플레이어 입력과 완전히 같은 타입을 낸다
act(state: SiegeState, rand: () => number): SiegeInput | undefined
```

그래서 "봇이 검증한 판 = 플레이어가 하는 판"이 성립한다. 만약 봇이 sim 내부를 직접 만지거나 별도 경로로 판정했다면, 검증 결과는 주장에 불과했을 것이다.

#### 3-2. 결정론이 왜 필수인가

- 같은 (시드, 입력 시퀀스) 면 항상 같은 결과 → 판정이 재현 가능하다.
- 봇이 남긴 커맨드 시퀀스가 **그대로 리플레이 포맷**이다. replay() 가 원본과 일치하는지 까지 테스트가 단언한다 (siege/test/bots.test.ts).

- 실제로 겪은 버그가 이 설계를 다듬었다: 침공 개시 때 `state.tick` 이 0으로 리셋되므로 커맨드 키를 tick으로 잡으면 준비 단계 커맨드가 침공 첫 틱에 덮인다. **키는 tick이 아니라 스텝 번호여야 한다.**

### 3-3. 단일 진실 원천

성 치수·충돌·높이는 sim의 `CASTLE` 상수 하나에서 나오고 렌더러가 그것을 읽는다. 렌더와 sim이 어긋나면 "봇이 검증한 판 = 플레이어가 하는 판"이 즉시 깨지기 때문이다.

## 4. 검증 파이프라인

`pnpm verify` — 봇 4등급 × 시드 6종 = 24판을 헤드리스로 돌려 판정하고 `siege/reports/latest.{md,json}` 에 리포트를 남긴다.

### 4-1. 봇 4등급 (하나로는 "성립한다"를 말할 수 없다)

| 등급                  | 하는 일                           | 답하는 질문                             |
|---------------------|--------------------------------|------------------------------------|
| <code>afk</code>    | 침공만 개시하고 <b>정말 아무것도 안 한다</b>   | 배치가 게임에 실제로 의미 있는가 ( <b>저야</b> 통과) |
| <code>deploy</code> | 표준 장착만 하고 전투 중엔 손대지 않는다        | 배치만으로 판이 성립하는가 (기준선)               |
| <code>greedy</code> | 병기를 전선으로 겨누고, 마법사를 옮겨 궁극으로 끓는다 | 플레이어의 개입이 <b>보상되는가</b>             |
| <code>random</code> | 장착 후 5초마다 30% 확률로 엉뚱한 곳에 겨누다   | 심사자가 대충 조작해도 안 무너지는가               |

`random` 이 **장착까지는 하고** 시작하는 이유가 있다. 장착제 도입 뒤에는 "아무것도 안 한 판"이 즉시 지는 것이 정상이라, 장착 없는 무작위 봇은 `afk` 와 같은 판정이 되어 **검증이 시늉**이 된다. 이 봇이 재는 것은 "게임에 들어온 사람이 엉뚱하게 만졌을 때"지 "게임을 시작 안 했을 때"가 아니다.

### 4-2. 판정 기준은 코드로 존재하고, 그 자체가 게임 설계 주장이다

`siege/bots/verify.ts` 의 `CRITERIA_V1` :

- 무개입은 **저야** 한다 → 장착(배치)이 게임에 의미가 있다는 뜻
- 표준 장착만으로는 **이겨야** 한다 (잔존 150~1400 / 2000) → 배치가 성립한다
- 표준 장착 여유가 **과하면 안 된다** → 이기는 게 당연하면 조작할 이유가 없다
- 적극 플레이가 표준 장착보다 **나아야** 한다 → 플레이어의 개입이 보상돼야 한다
- 장착 후 무작위 조작에도 **안 무너져야** 한다 → 심사자가 대충 만져도 성립해야 한다
- 판이 **45~150초**에 끝나야 한다 → 30~60초 플레이 영상에 담킨다
- 봇 커맨드 리플레이가 원본과 **일치**해야 한다 → 결정론

**V0 → V1: 기준을 바꾸는 것도 기획 결정이다.** V0은 첫 항목이 "무개입으로도 **이겨야** 한다"였다(기본 배치가 성립한다는 주장). 2026-08-08 「수비병 장착제」 도입으로 사용자가 이 전제를 뒤집었다 — *"아무것도 안 하면 지는 게 맞다. 그*

냥 이기면 게임의 의미가 없다." 그래서 무개입의 역할이 「성립의 증명」에서 「배치가 의미 있다는 증명」으로 바뀌었고, 구 기준선은 deploy (표준 장착 후 무개입)가 이어받았다. §5-5에 그 여정이 있다.

숫자를 맞추려고 기준을 느슨하게 하는 것과, 게임이 바뀌어 기준을 다시 정의하는 것은 다르다. 전자는 검증의 설득력을 깎고 후자는 검증을 살린다. 판정 항목이 **늘어난 것**(무개입 패배 단언이 추가됐다)이 그 차이의 증거다.

## 5. 증거: 실패 데이터와 통과까지의 경로

검증기가 진짜라는 증거는 통과 화면이 아니라 **반려 기록**이다. 2026-08-04 하루 동안 판정은 **0/6 → 5/6 → 6/6**으로 움직였고, 각 단계는 커밋으로 남아 있다.

### 5-1. 단계별 판정 이동 (2026-08-04, 기준 V0 시절)

| 단계 | 조치                   | 판정         | 남은 반려 사유                |
|----|----------------------|------------|-------------------------|
| 시작 | —                    | 0/6        | 무작위 조작에서 패배             |
| ①  | 위협 회피 유도 도입          | 0/6        | 무개입 패배 ×2, 적극 플레이 패배 ×4 |
| ②  | 궁수 폐지 → 대포 6         | 0/6        | <b>무개입이 너무 쉽다</b> ×4    |
| ③  | 웨이브 재조정              | 0/6        | 무작위 조작에서 패배 (유일)        |
| ④  | 대포 고정 + 조준 지정        | <b>5/6</b> | 무작위 조작에서 패배 ×1          |
| ⑤  | 병기 증원 + 벽 피해 하향·수 증가 | <b>6/6</b> | 없음                      |
| ⑥  | 보스 네크로맨서 추가          | <b>6/6</b> | 없음 (수치 재조정 후)           |

②에서 **실패의 방향이 뒤집힌 것**이 중요하다. "무개입 패배"에서 "무개입이 너무 쉽다"로 바뀌었다는 건, 판이 어려워져 못 이기는 문제에서 쉬워서 재미없는 문제로 옮겨갔다는 뜻이고, 그때부터는 난이도를 올릴 여지가 생긴다.

### 5-2. 최종 판정 (시드 6종 × 봇 4등급, 2026-08-09 출고분)

| 시드            | 무개입(afk)  | 표준 장착(deploy) | 적극 플레이(greedy) | 무작위(random) | 판정 |
|---------------|-----------|---------------|----------------|-------------|----|
| 20260725 (출고) | <b>패배</b> | 521           | 809            | 521         | 통과 |
| 1             | <b>패배</b> | 1074          | 1453           | 1074        | 통과 |
| 7             | <b>패배</b> | 773           | 938            | 773         | 통과 |
| 42            | <b>패배</b> | 567           | 1371           | 567         | 통과 |
| 777           | <b>패배</b> | 485           | 1097           | 485         | 통과 |
| 31337         | <b>패배</b> | 702           | 1412           | 702         | 통과 |

세 가지가 동시에 성립한다: - 무개입은 전 시드 패배 (43~44초에 함락) = 장착하지 않으면 성이 무너진다 → 배치가 게임이다 - 표준 장착만으로 전 시드 승리, 잔존 485~1074로 대역(150~1400) 안 = 배치가 성립한다 - 적극 플레이 > 표준 장착이 전 시드 성립 = 개입이 보상된다

판 길이는 114~119초로 기준(45~150초) 안이다. random 이 deploy 와 같은 수치인 시드가 있는 것은, 엉뚱한 조준이 결과를 바꾸지 못한 판이라는 뜻이다 — 무너지지도 않지만 이득도 없다. "대충 만져도 성립한다"가 정확히 이 모양이다.

### 5-3. 편성 비교 — 손감이 아니라 판정으로 고른다

로스터·배치·웨이브를 Loadout 데이터로 분리해, sim 코드를 건드리지 않고 편성을 갈아끼운다 (pnpm verify --compare ). 「유도」 도입 직후 측정:

| 편성                 | 무개입         | 적극 플레이     | 무작위 |
|--------------------|-------------|------------|-----|
| 구 출고 (궁수6·대포2·발리2) | 3/6 (잔존 92) | 1/6 (45)   | 0/6 |
| 궁수4·대포4·발리2        | 6/6 (1062)  | 4/6 (335)  | 0/6 |
| 대포6·발리2 (채택)       | 6/6 (1297)  | 6/6 (1372) | 0/6 |
| 궁수8·대포2            | 0/6         | 0/6        | 0/6 |

이 표가 궁수 폐지를 결정했다. 새 규칙 아래서 궁수는 적을 밀어내기만 하고 못 죽인다. "궁수가 약한 것 같다"가 아니라 "전 시드 전멸"이라는 숫자가 결정 근거였다.

### 5-4. 검증이 설계를 뒤집은 사례

사례 A — 보스가 너무 강해서가 아니라, 구조 때문에 못 잡혔다. 네크로맨서 초안(HP 3200 · 부활 쿨 7초 · 3기 · standoff 13)에서 무개입이 1/6로 무너졌다. 원인은 수치가 아니라 배치였다: 자동 조준은 가까운 적부터 치므로 뒤에 선 술사를 영영 못 끊고 부활이 무한히 돈다. standoff 를 13 → 6으로 당겨 성벽 화력 안에 들어오게 한 것이 "무개입으로도 이긴다"의 조건이었다. 후보 13종을 봇으로 돌려 최종안을 골랐다.

사례 B — 증원이 판정을 떨어뜨렸다. "성을 지키기엔 병기가 너무 적다"는 지시로 병기를 8 → 12로 늘렸더니 판정이 5/6 → 4/6으로 내려갔다. 웨이브를 올려 상쇄하려 하자 51기 4/6 → 61기 0/6이라는 절벽이 나왔다. 원인은 구조적이었다: 성벽 HP 가 공유 풀이라 벽에 붙은 수가 임계를 넘으면 초당 800+ 피해로 즉사한다. 개체당 벽 피해를 65% 낮추고 수를 34 → 62로 늘려 성벽이 완만하게 깎이도록 바꾸자 증원과 통과를 동시에 얻었다. 그제서야 "일부러 한 구간을 비우는" 유도 도박이 성립한다.

사례 C — 봇이 통과하는데 검증이 시늉이 될 뻔했다. 「대포 고정」을 넣자 random 봇의 이동 명령이 전부 무시돼 사실상 무개입과 같아졌다. 그대로 뒀다면 판정은 통과했겠지만 아무것도 검증하지 않는 통과였다. "대충 만진다"의 실체를 이동에서 엉뚱한 조준으로 바꿔 봇을 다시 썼다. 검증 대상이 바뀌면 검증기도 따라 바뀌어야 한다.

### 5-5. 판정 기준 자체를 바꾼 날 (2026-08-08 「장착제」)

플레이 피드백에서 시작해 게임의 전제와 검증 기준이 함께 뒤집힌 사례다.

1. 사용자가 화면을 보고 물었다 — "대포를 쏘는 병사는 그냥 대포에 붙어 있는 거야?" 조사해보니 병기마다 병사가 둘이었  
다: 7/27 시절의 장식 기사(sim에 없고 발사 모션도 없는 렌더 잔재)와 8/6 상주 조작제의 실제 수비병. 룰 개정 때 렌더

정리가 누락된 것이다.

- 2. 사용자가 룰을 바꿨다 — 병기는 **빈 채로** 시작하고 수비병을 보내 **장착**해야 쏜다.
- 3. 여기서 판정 기준과 충돌했다. 병기가 비어 있으면 무개입은 **반드시 진다**. V0의 첫 항목("무개입으로도 이겨야 한다")이 성립할 수 없다.
- 4. 사용자 결정 — "아무것도 안 하면 지는 게 맞다. 그냥 이기면 게임의 의미가 없다." → 기준을 **V1**으로 개정하고, 구 기준선은 `deploy` 봇을 새로 만들어 이어받게 했다.

**검증이 잡아낸 것들** (기준을 바꾼 뒤 6/6에 이르기까지):

| 증상                              | 실제 원인                                              | 조치                                                 |
|---------------------------------|----------------------------------------------------|----------------------------------------------------|
| 판이 준비 단계에서 안 끝남 (6기가 100초+ 공회전) | 집결한 수비병이 폭 2.6 경사로에 몰려 <b>서로 밀어내며</b> 못 오름         | 행군 중엔 겹침 분리를 끄는 <b>행군 관통</b> 규칙                    |
| 기준선이 614 → <b>113</b> 으로 붕괴     | 봇이 "장착 완료"를 순간 위치로 판정 → 지나가던 병사에 속아 <b>보행 중 개시</b> | 판정을 <b>의도 기준</b> (정지 장착 + 보행 목적지)으로                |
| —                               | —                                                  | → 재확인 시 <b>614/104.6초</b> , 구 상주 배치와 <b>정확히</b> 등가 |

마지막 줄이 이 개정의 결론이다. 장착제는 **밸런스가 아니라 시작 절차만** 바꿨다는 것이 숫자로 증명됐고, 그 등가가 테스트에 단언으로 박혀 있다.

## 5-6. 전제가 바뀌면 그 위의 수치는 전부 무효다 (2026-08-09)

같은 날 두 번, **어제 옳았던 판단이 오늘 틀린 값이 되는** 일이 일어났다. 검증이 없었다면 둘 다 모르고 지나갔을 것이다.

**사례 D — 마법사 사거리를 줄였다가 되돌렸다.** 측정 결과 마법사가 안뜰에 선 채로 성벽 36 중 30을 덮어 **움직일 이유가 없었다**. 사거리를 18 → 14로 줄여 재배치를 강제했다(6/6 유지). 그런데 그 뒤 "**마법사는 성벽 위에서 시전하는 병종**"이라는 디렉션으로 서는 위치를 보도 위로 옮기자 전제가 무너졌다 — 보도에서 벽 바깥면까지가 이미 7이라, 사거리 14 중 들판에 남는 건 7뿐이었다. 런타임 실측: 커서를 1·2·3·4시 어디에 두어도 레티클이 **13.9에** 몰려 괴수가 오는 방향으로 스킬을 쓸 수 없었다. 사거리를 18로 복구. 측정은 옳았지만 그 측정의 전제가 바뀌었다.

**사례 E — 근접 유닛이 일방적으로 맞고만 있었다.** 사용자 관찰 ("**병사들이 검으로 공격을 안 하네?**")에서 출발해 수치를 맞춰 보니: 괴수의 접전 거리는 자기 반경 + 아군 반경 + 0.9라 야귀 1.9·갑주귀 2.3인데, 수비병 사거리는 1.6(중심 기준)이었다. 적은 달고 병사는 못 닿는 구간이 존재했다. 공격이 적의 몸통에 닿도록(사거리 + 적 반경) 고치자 반격이 성립했고, 그만큼 난이도가 내려가 대포를 200 → 192로 재보정했다(6시드 스왑).

두 사례의 공통 교훈은 §6-4의 마지막 항목에 있다.

## 6. 주요 프롬프트 — 실제로 방향을 바꾼 지시들

전체 원문은 `docs/logs/YYYY-MM-DD.md`에 날짜별로 남아 있다. 아래는 **결과를 실제로 바꾼** 지시를 유형별로 추린 것이다. 길고 정교한 프롬프트가 아니라 **짧고 판정적인 한 문장**이 가장 큰 변화를 만들었다는 것이 이 프로젝트의 관찰이다.

6-1. 프레이밍을 바꾼 지시 (가장 효과가 컸다)

| 지시 (원문)                                           | 무엇이 바뀌었나                                                                                             |
|---------------------------------------------------|------------------------------------------------------------------------------------------------------|
| "선택지 자체가 마음에 안 든다"                                | 4지선다가 전부 "스텝 축 고르기"였음을 인지 → 한 단계 위 질문("플레이어가 내리는 재미 있는 결정은 무엇인가")으로 재설정. <b>배치 판단</b> 이라는 축이 여기서 나왔다 |
| "며칠째 예시를 줘도 형식이 바뀌는 게 없어"                         | "SC처럼"이라는 지시가 기존 UI에 덧붙여지고 있었다 → 작업 방식을 <b>레퍼런스를 스펙으로 재유도</b> 로 전환하고 기존 구현을 잔재로 취급                   |
| "코드 수정만 계속하지 말고 이유부터 찾아줄래"                        | 같은 표시 버그를 5회 고치던 반복을 끊었다 → 원인(높이를 무시하는 판정 = 원기둥) 도달 후 기능 자체를 폐지 (§7-6)                               |
| "우선 아무것도 안 하면 지는 게 맞지 않나? 그냥 이기면 게임의 의미가 없는 거 같아" | <b>판정 기준 V0 → V1</b> . 게임의 전제와 검증 기준이 함께 뒤집혔다 (§5-5)                                                 |

6-2. 규칙을 새로 만든 지시

| 지시 (원문)                                | 결과                                              |
|----------------------------------------|-------------------------------------------------|
| "대포 자체는 고정한다, 다만 쏘는 위치를 지정한다"          | 「대포 고정 + 조준」— 무작위 조작 반려가 <b>구조적으로</b> 사라졌다      |
| "시작하면 병기에 장착하는 형식. 장착하면 쏘고 아니면 분리"     | 「수비병 장착제」— 배치가 게임의 첫 수가 됐다                      |
| "E는 궁극기, 전사는 공격기만, 마법사는 속성 통일, 성주는 버프" | 스킬 3×3 체계. 초안을 그대로 받지 않고 <b>슬롯의 의미</b> 를 지정한 지시 |
| "홀드를 안 누르면 자동 공격해야 할 것 같은데"            | 자동 교전 + <b>holding</b> 상태. 사용자의 표현이 그대로 규칙이 됐다  |
| "보스는 마지막 웨이브에 같이 성벽으로 이동해야 할 것 같다"     | 보스 속도·등장 시점 재조정 (6시드 스위프로 값 선택)                 |

6-3. 화면을 보고 결함을 짚은 지시

이 유형은 **SI**가 스스로 발견하지 못한 것들이다. 코드는 의도대로 돌고 있었기 때문이다.

| 지시 (원문)                       | 실제 원인                                                             |
|-------------------------------|-------------------------------------------------------------------|
| "대포를 쏘는 병사는 그냥 대포랑 붙어 있는 거야?" | 병기마다 병사가 둘 — 렌더 잔재(장식)와 실제 유닛이 공존                                 |
| "갑옷을 동일하게 입고 있네?"             | 색만 바꾼 같은 모델 → 마법사·성주를 <b>별도 조형</b> 으로 분리                          |
| "칼이 아니라 뿔 던지는데?"              | 전사가 사거리 13 검기 투사체를 쏘고 있었다 → 근접화. <b>벽 뒤에서 공짜로 벌던 화력</b> 이 함께 드러났다 |
| "이렇게 멍하니 있다니까"                | 괴수 접전 1.9 < 병사 도달 2.2의 사각 → 자동 교전 도입                              |
| "1~3시 방향에 스킬을 못 쓴다"           | UI가 아니라 <b>사거리</b> 문제 — 성벽 위에서 들판으로 7밖에 못 나갔다                     |
| "보스를 잡은 기억이 없는데"              | 보스가 늘 마지막에 죽고 그 죽음이 곧 판 종료였다(위협 지속 8초)                            |



## 6-4. 프롬프트에서 배운 것

- **판정 가능한 지시가 이긴다.** "예쁘게"는 매번 다른 결과를 낳았고, "아무것도 안 하면 져야 한다"는 코드로 옮겨져 테스트가 됐다.
- **AI에게 "재서 보여줘"를 시키면 논쟁이 끝난다.** 추측 다섯 번보다 실측 한 번이 싸다. 이 문서의 표 대부분이 그렇게 나온 숫자다.
- **화면 확인은 사람이 해야 한다.** §5-3의 여섯 건은 전부 코드가 "의도대로" 도는 상태에서 사람이 화면을 보고 짚은 것이다. AI는 자기 의도와 화면의 불일치를 잘 못 본다.

## 7. 디렉팅 기록 — 반려와 재시도의 원문 사례

전체 원문은 `docs/logs/YYYY-MM-DD.md` 에 있다. 아래는 **AI의 산출을 사람이 반려하고 방향을 바꾼** 대표 사례다. 이 프로젝트에서 사람의 일은 코드를 쓰는 것이 아니라 **무엇이 틀렸는지 판정하고 프레이밍을 바꾸는** 것이었다.

### 7-1. 선택지의 고도를 반려한 사례 (2026-08-04)

- 상황: 궁수 역할을 정하기 위해 "관통 / 연사↑ / 상성 / 폐지" 4지선다를 제시
- 지시: "선택지 자체가 마음에 안 든다"
- 판단: 네 선택지가 전부 "스탯 축 하나 고르기"였다. 그건 밸런스 조정이지 기획이 아니다.
- 조치: 한 단계 위 — "이 게임에서 플레이어가 내리는 재미있는 결정이 무엇이어야 하나"로 재질문. 거기서 **배치 판단**이라는 축이 나왔고, 병종 구성·괴수 특징·밸런스가 전부 그 밑으로 따라왔다.
- 교훈: 하위 항목을 못 정하는 건 상위 축이 안 정해졌기 때문일 때가 많다.

### 7-2. 사람이 제3의 설계를 낸 사례 (2026-08-04)

- 상황: 재배치 비용 룰로 4지선다(이동 중 사격 허용 / 속도 상향 / 준비 시간 / 보도 보정)를 제시
- 지시: "대포 자체는 고정한다, 다만 쏘는 위치를 지정한다. 날아가는 경로를 보이게 해 거리에 따라 날아가는 시간을 별도로 한다. 병사 일부는 몬스터가 침범했을 때 백병전 인력으로 변동할 수 있다"
- 결과: 선택지 어느 것도 아닌 새 설계. **무작위 조작 반려가 구조적으로 사라졌다** — 대포를 못 옮기니 잘못된 명령이 대포를 침묵시킬 수 없다.
- AI가 지적한 충돌: 대포가 고정이면 구간별 위협이 균일해져 플레이어가 화망을 못 본다. → **위협을 위치가 아니라 조준에서 계산하는** 안을 제시해 해소. 사용자 승인.

### 7-3. AI의 사실 오류를 코드로 잡은 사례 (2026-08-04)

- 상황: "괴수가 4면으로 온다"는 전제로 침공 방향을 질문
- 조치: 질문 전에 실제 코드를 확인 → **동쪽 1면에서만 왔다**(`world.ts` 스폰이 x 고정, z만 산개). 4면 개방은 **플레이어 보**도 이야기였다. 즉 "세미 3면"은 사용자가 생각한 축소가 아니라 확장.
- 조치: 사실을 정정해 알린 뒤 다시 질문 → 「정면 전폭 + 모서리 회절」로 확정.
- 교훈: 기획 질문을 던지기 전에 현재 코드가 실제로 뭘 하는지 확인할 것. 잘못된 전제 위의 선택은 통째로 되돌려야 한다.

7-4. 일정 오해를 사람이 지적한 사례 (2026-08-04)

- 상황: AI가 마일스톤 문서의 날짜 배분(8/7~8/9 제출)을 마감처럼 취급해 "개발 가능일 사흘"로 계산
- 지시: "마감일은 10일인데 어디서부터 오해가 생긴 걸까?"
- 판단: 그 배분은 "기획이 8/3에 끝난다"는 전제로 짠 것이었다. 전제가 깨졌으면 배분을 다시 짜야 한다.
- 조치: 제출 작업 실측 재산정(1.5~2일) → 개발 가능일 **나흘**로 정정.

7-5. 그 외 반려 기록 (요약)

| 날짜    | 반려 내용                             | 조치                                                                            |
|-------|-----------------------------------|-------------------------------------------------------------------------------|
| 07-25 | "다크소울"을 분위기로 해석                   | 렌더링 퀄리티 지칭임을 재정정                                                              |
| 07-25 | "달라진 게 없다"                        | 기본 화면 프레이밍 자체를 수정                                                             |
| 07-27 | "밤이 생각보다 안 보인다"                   | 밤 씬 폐기, 낮 씬으로 전환                                                              |
| 08-01 | 피격 플래시가 분홍 사탕색                    | 따뜻한 백색으로 교체 (붉은 피부 + 순백 emissive 문제)                                          |
| 08-06 | "기존 디자인에 억지로 넣은 느낌 · 기본이 안 되어 있다" | 어택땅·정지를 sim 커맨드로, 하단 바 전폭 재설계                                                 |
| 08-07 | "며칠째 예시를 줘도 형식이 안 바뀐다"            | 작업 방식 전환 — 레퍼런스를 스펙으로 <b>재유도</b> , 기존 구현은 잔재로 취급                              |
| 08-09 | "갑옷을 동일하게 입고 있네"                  | 색 스왑을 폐기하고 <b>실루엣을 분리</b> — 마법사 전신 로브, 성주 갑옷 없는 귀족 예복                         |
| 08-09 | "칼이 아니라 뭘 던지는데?"                  | 전사가 사거리 13 겹기 투사체를 쏘고 있었다 → 근접(2.4)으로. <b>벽 뒤에서 공짜로 벌던 화력</b> 이 함께 드러나 대포로 보정 |
| 08-09 | "이렇게 멍하니 있다니까"                    | 근접 유닛 <b>자동 교전</b> 도입(홀드 시 정지). "자동 추격 없음은 설계"라던 이전 답을 정정                     |

7-6. 표면을 다섯 번 고치고서야 원인에 닿은 사례 (2026-08-09)

이 문서에서 가장 값어치 있는 실패 기록이다.

스킬 사거리 표시가 이상하다는 지적에 **다섯 번**을 고쳤다 — 가림 판정을 꺾다가(벽을 뚫고 그려짐), 지형을 따라 그렸다가(성벽 위 시전자 기준으로 원이 지면에 내려앉아 중심이 어긋남), 시전자 평면으로 올렸다가(공중에 뜬 띠로 보임)… 매번 스크린 샷에 보이는 상황 하나만 고쳤고, 그때마다 **다른 조합에서 깨졌다**.

사용자가 반복을 끊었다 — "코드 수정만 계속하지 말고 **이유부터 찾아줄래?**"

그제야 원인에 닿았다. **sim의 사거리 판정은 높이를 무시한다**(XZ 평면 거리만 잴다). 즉 시전 가능 영역은 원이 아니라 **수직 원기둥**이고, 원기둥을 화면에 그리려면 어느 높이의 단면을 그릴지 골라야 하는데, 이 판에는 높이 0(지면)과 11(성벽)이 섞여 있어 어느 단면을 골라도 **다른 상황에서 깨진다**.

결론은 원을 **폐지**하는 것이었다. 사거리는 레티클이 경계에서 멈추는 것으로 알리고 (경계에 물리면 색이 바뀐다), 시전자와 착탄점을 선 하나로 잇는다. 표시할 수 없는 것을 표시하려던 시도 자체를 접자 그 계열의 버그가 소멸했다.

**교훈:** 증상이 반복되면 고치는 속도를 올릴 게 아니라 멈춰야 한다. 그리고 조합(시전자 높음/낮음 × 대상 지형 높음/낮음)을 **먼저 열거**했어야 했다. 이 교훈은 §5-6의 두 사례와 같은 뿌리다 — **전제를 적어두지 않으면 그 위의 판단이 조용히 썩는다.** | 08-04 | "보스·스케일·질감을 왜 버리는 거지?" | 순위는 "자를 목록"이 아니라 "밀릴 때 뒤에서부터 밀리는 순서"임을 명확화 |

## 8. 자체 검증 루프 — AI가 자기 결과물을 확인하는 방법

### 8-1. sim 단언이 스크린샷보다 우선

`siege/test/world.test.ts` (vittest, 89 테스트)가 렌더 없이 단언한다: 결정론 · 무방비 패배 · 기본 배치 방어 성공 · 유도 편향 · 지연 규칙 · 회절 정지선 · 고정 병기 불이동 · 보스 부활/시체 소모/삭음.

교훈(로그 원문): "성공 스크린샷은 우연일 수 있다 — sim 단언이 먼저"

### 8-2. 실기기 브라우저 점검에서만 잡히는 것

헤드리스로는 못 잡고 실제 GPU 브라우저에서만 드러난 결함들:

- **조준선이 아예 안 보였다** — `depthTest` 가 켜져 있어 안돌 시점에서 선이 성벽 뒤로 사라졌다. 화망이 안 보이면 유도 규칙 자체가 안 읽힌다. 지휘 오버레이이므로 `depthTest`를 꺾다.
- **고정 병기가 밀려나 있었다** — 대포가 성문 발리스타와 반경이 겹쳐 배치 좌표가  $\pm 3 \rightarrow \pm 3.22$ 로 조용히 어긋나 있었다. "고정"이라는 규칙과 모순.
- **피격 플래시 색** — 정지 화면 한 장으로 바로 드러났다.

### 8-3. 관측 환경의 함정 (재현 시 참고)

- `pnpm dev | head -20` 로 띄운 dev 서버가 SIGPIPE로 죽어 페이지가 예셋 없이 새까맣게 렌더됐다. 코드 문제가 아니었다 — 백그라운드 실행 시 파이프로 자르지 말 것.
- 브라우저 탭이 백그라운드면 `document.hidden=true` 라 rAF가 멈춰 sim이 흐르지 않는다.
- 헤드리스 GPU(swiftshader)는 0.5fps라 실시간 대기로는 sim이 안 흐른다 → `window.__siege.fastForward(n, input?)` 혹은 감고 촬영한다.

### 8-4. 측정 없이 최적화하지 않는다

성능 병목을 fps로만 보면 원인(드로우콜인지 fill-rate인지)을 못 가른다. `__siege.perf()` (renderer.info 프레임 누적)로 재서 **병목이 fill-rate가 아니라 드로우콜**임을 확인한 뒤 대책 3종(리그 정적 병합 · 소품 인스턴싱 · 그림자 캐스터 축소)을 적용했다. 결과: 전투 1240 → 785 드로우콜, 16.2ms ≈ 62fps (개발기 GPU = Intel UHD 630).

이 수치는 적 34기·병기 8기 시절 측정값이다. 이후 콘텐츠가 늘어(최대 동시 괴수 24기 · 병기 12기 · 보스) 재측정이 필요하다. 자동화로 띄운 백그라운드 탭은 rAF가 멈춰 측정값이 0으로 나오므로, 포그라운드 탭에서 다시 재야 한다 — 제출 전 갱신 항목.

## 9. 외부 에셋·오픈소스 출처

### 9-1. 외부 에셋 — 전부 CC0 (저작자 표기 불요, 상업 이용 가능)

| 에셋                               | 용도           | 출처         |
|----------------------------------|--------------|------------|
| Kloppenheim 02 (Puresky) HDRI 1K | 환경광·반사       | Poly Haven |
| Bricks075A 1K                    | 성벽 석재 PBR    | ambientCG  |
| Grass004 1K                      | 들판 PBR       | ambientCG  |
| Ground048 1K                     | 흙길 PBR       | ambientCG  |
| PavingStones128 1K               | 안뜰 포석 PBR    | ambientCG  |
| Rock Moss Set 01 (GLTF 1K)       | 들판 바위 (포토스캔) | Poly Haven |
| Dead Tree Trunk (GLTF 1K)        | 고사목 (포토스캔)   | Poly Haven |
| Wooden Crate 01 (GLTF 1K)        | 안뜰 께짚        | Poly Haven |

전체 목록·URL은 docs/asset-licenses.md . 에셋 추가 시 그 자리에서 기록하는 것을 프로젝트 규칙(CLAUDE.md)으로 두었다.

### 9-2. 코드로 생성한 것 (외부 에셋 아님)

- **성채·지형·소품 지오메트리**: three.js 절차 조합 ( client/src3d/environment.ts )
- **캐릭터·괴수·보스 모델 전량**: 절차 지오메트리 ( client/src3d/models.ts ) — 폴아머 기사, 언데드 3종(야귀·질주귀·갑주귀), 네크로맨서, 공성 병기(대포·발리스타)
- **입자 FX**: 고정 풀 파티클 ( client/src3d/particles.ts ) — 연기 48 · 파편 96
- **사운드 전량**: WebAudio 실시간 합성 ( client/src3d/audio.ts ) — **오디오 파일 0개**. 오실레이터+필터드 노이즈로 13종 합성, 카메라 기준 거리 감쇠·스테레오 패닝. 외부 에셋·라이선스·번들 크기가 전부 0이라 시각 요소와 같은 원칙이 유지된다.
- **톤 그레이딩**: 비네트 셰이더 패스에 스플릿 톤·대비·채도를 통합 (패스 추가 0)
- **폰트**: 시스템 폰트. 번들 폰트 없음.

### 9-3. 오픈소스 의존성

| 이름         | 버전    | 용도                                                                  | 라이선스       |
|------------|-------|---------------------------------------------------------------------|------------|
| three.js   | 0.185 | 3D 렌더링                                                              | MIT        |
| TypeScript | 7.0   | 언어                                                                  | Apache-2.0 |
| Vite       | 8.1   | 번들러·개발 서버                                                           | MIT        |
| vitest     | 4.1   | 테스트 러너                                                              | MIT        |
| tsx        | 4.23  | TS 실행 (봇 검증 CLI)                                                    | MIT        |
| Phaser     | 3.90  | <b>v5에서는 미사용</b> — 2D 시절(v1~v4) 잔여 의존성. <code>legacy-2d</code> 브랜치용 | MIT        |

런타임에 유료 API·라이선스를 사용하지 않는다 (심사 요건). AI 활용은 전부 빌드 타임이며, 빌드 산출물은 정적 파일만으로 동작한다.

## 10. 재현 방법

```
pnpm install
pnpm dev      # localhost:5173 — 우클릭: 병기 조준 / 이동, 휠 줌, Space 침공
pnpm test     # sim·봇 단언 89개
pnpm verify   # 봇 4등급 × 시드 6종 → siege/reports/latest.md
pnpm verify --compare  # 편성별 비교표
pnpm typecheck && pnpm build
```

- 배포: GitHub Pages (심사자가 링크 클릭만으로 플레이)
- 커밋 히스토리가 제출물의 일부다 — 각 판정 이동이 커밋 단위로 남아 있다.